

# JARVIS

Personal AI Assistant — Build Guide · Phase 1 to 3

|          |                                                   |
|----------|---------------------------------------------------|
| Stack    | Python · Whisper · Piper TTS · Ollama · SQLite    |
| Trigger  | Hotkey (press key → speak → JARVIS responds)      |
| Privacy  | Fully offline — no API keys, no internet required |
| Hardware | 16 GB RAM, no GPU — all tools chosen for CPU      |
| OS       | Arch Linux + Hyprland · Neovim + LazyVim          |

## Phase 1

### Python Foundation + Project Setup

Weeks 1–4

#### GOAL

Solidify Python to the point where you can write clean modules confidently. By the end, you have a working CLI study tracker — JARVIS's first real component.

#### 1.1 Dev environment

|                     |                                                                                                                                            |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Install uv          | Search: 'uv python package manager install arch linux'. Use it instead of pip + venv for all projects.                                     |
| Init JARVIS project | uv init jarvis → creates pyproject.toml, .venv, src/ layout automatically.                                                                 |
| Neovim Python LSP   | Search: 'LazyVim python pyright ruff setup'. Add pyright + ruff-lsp to your config. These give you autocomplete and linting inside Neovim. |
| Project structure   | jarvis/ → core/ voice/ brain/ study/ utils/ main.py — one folder per phase module.                                                         |

#### 1.2 Python concepts to learn

|                  |                                                                                                                        |
|------------------|------------------------------------------------------------------------------------------------------------------------|
| OOP in Python    | Search: 'Python dataclasses vs classes', 'dunder methods Python'. Your C++ OOP maps directly — syntax is just lighter. |
| Decorators       | Search: 'Python decorators practical examples'. You'll use these for hotkey bindings and logging.                      |
| Context managers | Search: 'Python with statement context manager'. Critical for file and audio stream handling.                          |
| asyncio basics   | Search: 'Python asyncio beginner tutorial'. JARVIS listens and processes simultaneously — needs async.                 |
| pathlib + JSON   | Search: 'Python pathlib tutorial', 'Python json load dump'. Used everywhere for reading config and notes.              |

#### 1.3 SQLite — JARVIS memory

|                                |                                                                                                                                       |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Learn sqlite3</b>           | Search: 'Python sqlite3 tutorial CRUD'. Built into Python, zero install.                                                              |
| <b>Design your schema</b>      | Tables: sessions(date, subject, duration_min), topics(name, subject), quiz_scores(topic, score, date). This powers the study tracker. |
| <b>Build study tracker CLI</b> | Commands: jarvis start / jarvis stop / jarvis report. Stores data in SQLite. This is Phase 1's final project.                         |

***Tip** Python's biggest difference from C++ is that everything is an object and types are inferred. Stop declaring types everywhere — embrace it. Add type hints (search: 'Python type hints cheat sheet') only where it helps readability.*

## Phase 2 Voice Pipeline

Weeks 5–9

### GOAL

Press a hotkey → speak → JARVIS transcribes your words and speaks a response back. Fully offline. This is the core interaction loop everything else builds on.

### 2.1 Audio capture

|                             |                                                                                                                              |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Install sounddevice</b>  | Search: 'sounddevice python record microphone'. Records mic audio as a numpy array. Simple and lightweight.                  |
| <b>Test mic input</b>       | Write a script that records 5 seconds on keypress, saves as .wav, plays back. Confirm audio capture works before adding STT. |
| <b>Handle audio devices</b> | Search: 'sounddevice list devices python'. On Hyprland/PipeWire you may need to set the correct device index.                |

### 2.2 Speech → Text (Whisper, fully offline)

|                              |                                                                                                                                               |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Install Whisper</b>       | Search: 'openai whisper install python'. Run: pip install openai-whisper. No internet needed after install — model runs locally.              |
| <b>Choose model size</b>     | Use 'base' model — good accuracy, fast on CPU (2–4 sec per utterance). Search: 'whisper model sizes comparison'. Avoid 'large' without a GPU. |
| <b>Transcribe audio</b>      | Load model once at startup. Pass .wav file → get text string. Search: 'whisper python transcribe example'.                                    |
| <b>Whisper + sounddevice</b> | Combine: record on hotkey press → save temp .wav → whisper.transcribe() → get text. This is your voice input module: voice/listener.py        |

### 2.3 Hotkey trigger

|                       |                                                                                                                        |
|-----------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Install pynput</b> | Search: 'pynput keyboard listener python'. Captures global keypresses even when terminal is not focused.               |
| <b>Bind a hotkey</b>  | Map e.g. Super+J to start recording. Hold key → record. Release key → transcribe. Search: 'pynput hold key to record'. |

### Hyprland keybind

Alternatively bind the hotkey in `hyprland.conf` to run a Python script. Search: 'hyprland bind exec keybind'. Either approach works.

## 2.4 Text → Speech (Piper TTS, fully offline)

### Install Piper

Search: 'piper tts install linux'. Download the binary + an English voice model. Sounds more natural than `pyttsx3`.

### Run Piper from Python

Use subprocess to call piper binary: `echo 'text' | piper --model voice.onnx --output_file out.wav`. Then play `out.wav` with `aplay` or `python-vlc`.

### Wrap in `speak()` function

Create `voice/speaker.py` with `speak(text)` → generates wav → plays it. JARVIS will call this function everywhere.

## 2.5 Phase 2 milestone

- Press hotkey → say 'hello JARVIS' → JARVIS speaks back 'Hello, I heard you say: hello JARVIS'
- Both `listener.py` and `speaker.py` exist as clean modules
- No hardcoded paths — use `pathlib` and a `config.json`

*Tip Piper TTS with the 'en\_US-lessac-medium' voice sounds noticeably better than other options. Search: 'piper tts voices download' to browse available models.*

## Phase 3

### Local LLM Brain

Weeks 10–15

#### GOAL

JARVIS now understands what you're asking and gives intelligent responses — not just echo. The LLM runs entirely on your machine via Ollama. By end of phase: full voice conversation with short-term memory.

## 3.1 Install and run Ollama

### Install Ollama

Search: 'ollama install arch linux'. Runs as a local server on port 11434. Pull a model: `ollama pull llama3.2:3b`

### Choose your model

Start with `llama3.2:3b` (~2GB, fast). Upgrade to `mistral:7b-instruct-q4` (~4.5GB) once stable. Search: 'best ollama models for CPU only'.

### Test via curl

Search: 'ollama API curl example'. Send a POST to `localhost:11434/api/chat` — confirm you get a response before writing Python code.

## 3.2 Connect Python to Ollama

### Use requests library

Search: 'ollama python requests API'. POST to `/api/chat` with messages list. Returns JSON with the model's reply. No SDK needed.

### Stream responses

Search: 'ollama streaming python'. Set `stream=True` to get words as they arrive — lets Piper start speaking before the full reply is done.

|                     |                                                                                                                                              |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Create brain/llm.py | Function: ask(prompt, history=[]) → returns reply string. history is a list of {role, content} dicts — pass the full conversation each call. |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------|

3.3 System prompt — JARVIS persona

|                          |                                                                                                                                                                                         |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Write a system prompt    | Search: 'LLM system prompt best practices'. Tell the model: name is JARVIS, you are a CS student assistant, be concise, speak in plain English suitable for voice output (no markdown). |
| No markdown in responses | Critical: TTS will speak asterisks and hashes aloud. Add to system prompt: 'Never use markdown. Use plain sentences only.'                                                              |
| Tune for your degree     | Include in system prompt: your current subjects, your university level. JARVIS then gives CS-relevant examples automatically.                                                           |

3.4 Conversation memory

|                         |                                                                                                                                                                                  |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Short-term memory       | Keep last N turns in a Python list. Pass full list to Ollama each request. N=10 is a good start — fits in the context window easily.                                             |
| Summarize old turns     | When history grows beyond N turns, ask the LLM to summarize the older ones into one paragraph. Prepend summary to new history. Search: 'LLM conversation summarization pattern'. |
| Persist across sessions | Save history list to SQLite at end of each session. Load on startup. JARVIS remembers what you discussed yesterday.                                                              |

3.5 Wire everything together — main.py

|                    |                                                                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Import all modules | from voice.listener import listen / from voice.speaker import speak / from brain.llm import ask / from utils.memory import load_history, save_history |
| Main loop          | on_hotkey_press → listen() → transcribe → ask(text, history) → speak(reply) → append to history → loop. asyncio keeps hotkey listener non-blocking.   |
| Graceful shutdown  | Catch KeyboardInterrupt → save_history() → speak('Goodbye') → exit. Search: 'Python asyncio graceful shutdown'.                                       |

3.6 Phase 3 milestone — JARVIS v0.1

- Press hotkey → 'What is the difference between a process and a thread?' → JARVIS answers clearly
- Ask a follow-up → JARVIS remembers the context from the previous question
- Restart JARVIS → it still remembers the previous session
- All local — no internet, no API key, no cost

*Tip If Ollama responses feel slow (> 8 sec), switch to llama3.2:3b with 'num\_ctx': 2048 in the request body. Smaller context = much faster on CPU. You don't need a large context for voice conversation.*

QUICK REFERENCE

| Tool | Purpose | Install / Search term |
|------|---------|-----------------------|
|------|---------|-----------------------|

|                |                               |                                   |
|----------------|-------------------------------|-----------------------------------|
| uv             | Python package + venv manager | search: 'uv python install'       |
| sqlite3        | JARVIS memory / study data    | built into Python stdlib          |
| sounddevice    | Mic audio capture             | pip install sounddevice           |
| openai-whisper | Speech → text (offline)       | pip install openai-whisper        |
| piper          | Text → speech (offline)       | search: 'piper tts install linux' |
| pynput         | Global hotkey listener        | pip install pynput                |
| ollama         | Run local LLMs (no GPU)       | search: 'ollama install arch'     |
| llama3.2:3b    | Fast LLM for CPU              | ollama pull llama3.2:3b           |
| requests       | Call Ollama REST API          | pip install requests              |

## PROJECT STRUCTURE

```

jarvis/
core/
  config.json # paths, model names, hotkey binding
voice/
  listener.py # sounddevice + whisper → text
  speaker.py # piper TTS → plays audio
brain/
  llm.py # ollama API client + streaming
  memory.py # conversation history + SQLite
study/
  tracker.py # session timer, subject log
  db.py # sqlite3 schema + queries
utils/
  hotkey.py # pynput listener
  main.py # async event loop, wires all modules
pyproject.toml # uv project config

```

After Phase 3 you have JARVIS v0.1 — a real, working, fully local voice assistant. Phase 4 adds the study module (PDF reading, quizzing, progress reports). Phase 5 adds PC automation (window control, browser tasks). Phase 6 adds a proper agent loop with tool calling and a vector knowledge base.